ABSTRACT

Data structure synthesis

CALVIN LONCARIC, University of Washington, Seattle, WA, United States

Published: 01 November 2016

FSE'16: 24nd ACM SIGSOFT International Symposium on the Foundations of Software Engineering
November 13 - 18, 2016
WA, Seattle, USA

Conference Sponsors:
SIGSOFT

FSE 2016: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (November 2016)
https://doi.org/10.1145/2950290.2983946
ISBN: 9781450342186

Data Structure Synthesis

Calvin Loncaric
University of Washington, USA
loncaric@cs.washington.edu

ABSTRACT

All mainstream languages ship with libraries implementing lists, maps, sets, trees, and other common data structures. These libraries are sufficient for some use cases, but other applications need specialized data structures with different operations. For such applications, the standard libraries are not enough.

I propose to develop techniques to automatically synthesize data structure implementations from high-level specifications. My initial results on a large class of collection data structures demonstrate that this is possible and lend hope to the prospect of general data structure synthesis. Synthesized implementations can save programmer time and improve correctness while matching the performance of handwritten code.

CCS Concepts

•Software and its engineering → Automatic programming; •Theory of computation → *Data structures design and analysis;*

Keywords

Program Synthesis, Data Structures, Automatic Programming

1. INTRODUCTION

Data structures hide implementation details and offer programmers a convenient abstraction to program against. However, creating abstractions is difficult: programmers spend a great deal of effort designing, implementing, and debugging data structure implementations. Even standard library data structure implementations—with far simpler interfaces than many complex application-specific data structures—suffer from bugs. The issue tracker for libstdc++ (a popular implementation of the C++ standard template library) lists hundreds of issues related to their data structure implementations spanning more than a decade [8]. Model checkers have even found bugs in the Rust standard library [16], despite its memory protections. In my own work I have found that application-specific data structures are rarely implemented correctly at first and require a great deal of manual effort to debug [9].

FSE'16, November 13–18, 2016, Seattle, WA, USA
ACM. 978-1-4503-4218-6/16/11...$15.00
http://dx.doi.org/10.1145/2950290.2983946

```
Graph
    nodes : Set<Int>
    edges : Set<{n1 : Int, n2 : Int}>

    connected_edges(n : Int) =
        [ e | e ← edges, e.n1 == n || e.n2 == n ]

    degree(n : Int) =
        len(connected_edges(n))
```

Figure 1: Graph data structure specification describing abstract state (`nodes` and `edges`) and operations over that state (`connected_edges` and `degree`). The synthesizer decides what concrete information to store and efficiently implements `connected_edges` and `degree`.

I propose to synthesize complete implementations of specialized, application-specific data structures from high-level specifications. Figure 1 shows a sample specification for a graph data structure. The graph specification is much simpler than an implementation since it does not describe internal implementation details, only how each function should behave in terms of the data structure's abstract state. My hope is that such a synthesis tool could save programmer time and improve the correctness of the software they write without impairing performance.

2. BACKGROUND

Modern synthesis techniques have demonstrated great potential to save human and computer time. Thus far they have primarily seen success in specialized domains such as optimizing bit-vector programs [7] and deriving general string transformations from examples [5]. These techniques should be extended beyond program snippets to whole class implementations. Data structures offer a clean abstraction boundary, enabling a synthesizer to decide on internal implementation details and generate code without human intervention.

Automatic data structure implementation had its beginnings with iterator inversion [3] and its follow-ups [4, 11]. Iterator inversion used manually constructed rewrite rules to transform set comprehensions into optimized data structures.

The rules are difficult to write and even more difficult to prove exhaustive. The rewrite engine is fairly naive, so performance gains are not guaranteed. There was also work on automatic representation selection for the SETL language [14, 13, 2]. These techniques performed complex source code analyses to bound the possible contents of each data structure; the bounds could then be used to choose a good representation. However, these algorithms could only generate more efficient set or map implementations; data structures with more complex interfaces remained elusive.

More recent work can synthesize data structures from relational logic specifications [6]. The RelC synthesis tool works by exhaustively enumerating candidate data structure representations. A query planner then determines how to use each representation to implement the data structure's methods. Each candidate implementation is evaluated using an auto-tuning benchmark, and the best one is returned to the programmer. Since the RelC planner is opaque, the tool cannot rule out candidate representations quickly and relies entirely on benchmarking to select a good representation. In contrast, our techniques can optimize for a coarse cost model to guide the search toward better implementations faster. Furthermore, specifications are limited to collections that query their contents for conjunctions of equalities (e.g. `elem.id == N && elem.age == M`). Our work lifts this restriction and allows queries containing disjunctions (as in `connected_edges` in Figure 1) and negations.

3. CURRENT PROGRESS

I have built a tool called Cozy that can synthesize collections with add, remove, update, and query routines [9]. Cozy does not use manually constructed rewrite rules and can guarantee an optimal result with respect to a cost model. Generating an optimal implementation directly from a specification such as that of Figure 1 would be intractable due to state space explosion. A key insight of my work is to break up this task into two simpler steps. First, Cozy generates an "outline" or high-level plan, expressed in an intermediate language I developed for describing data structure implementations. Second, Cozy generates the implementation from the outline. The intermediate language has a *small-model property*: verifying a candidate implementation in the language with respect to the specification only requires solving a very simple formula.

Since a cheap verification procedure exists, my technique can find an intermediate representation program for a given data structure specification using counterexample-guided inductive synthesis (CEGIS) [15]. I also extended the traditional CEGIS algorithm to optimize solutions for a simple cost model, helping the tool converge on good data structure implementations quickly (within 90 seconds).

To evaluate Cozy's effectiveness, I used it to synthesize replacements for core data structures in four real-world programs: Myria [10] (a distributed database), ZTopo [17] (a topological map viewer), Bullet [1] (a real-time physics simulation library), and Sat4j [12] (a boolean satisfiability solver). For each program I replaced one important data structure. The four data structures have very different implementations and purposes, demonstrating Cozy's wide applicability: Myria stores analytics data indexed by ID and by time range, ZTopo caches map tiles organized by access time and state (in memory, on disk, or available over network), Bullet tracks bounding boxes for fast intersection testing, and Sat4j keeps organized data for each variable in the formula.

I compared the handwritten and synthesized implementations along three dimensions:

- **Correctness:** Across the four case studies there were 23 correctness bugs reported in their issue trackers for the handwritten data structures. Cozy's implementations are correct by construction, and do not suffer from these bugs.

- **Programmer effort:** The specifications for the data structures ranged from 10 to 25 lines long, while the original implementations ranged from 269 lines (Myria) to over 2500 (Bullet). The synthesizer finds good implementations for these in less than 90 seconds in all cases.

- **Performance:** On real-world benchmarks, the synthesized implementations performed within 5–10% of the original implementations. In one case study (Myria), the synthesized implementation was asymptotically better than the original implementation and ran orders of magnitude faster.

4. PROPOSED WORK

Cozy is aimed at data structures for efficiently retrieving subsets of a single collection. Many real-world data structures track multiple collections at once (such as the nodes and edges in a graph data structure) and aggregations over the entire collection (such as the smallest enclosing volume of a set of spatial objects). To be more applicable in practice, I will need to extend my synthesizer to handle these use cases as well.

To address this problem, I plan to take two steps: (1) extend Cozy to be able to retrieve aggregations over collections and (2) find ways to decompose complex data structure specifications into smaller pieces that Cozy can synthesize.

The first can be accomplished by adding additional primitives to Cozy's library; since Cozy already knows how to maintain indexes over subsets of elements, it could be extended to track sums, minimums/maximums, and other aggregations over those subsets instead.

The second will require a carefully chosen set of rules for breaking apart a complex specification into queries over single collections. For example, a comprehension of the form

```
[_ | x ←S1, y ←S2, y > 10 && y > x]
```

can be implemented using nested loops:

```
for y in [ y | y ←
S2, y > 10 ]:
    for x in [ x | x ←
S1, x < y ]:
        ...
```

While the original comprehension is not something Cozy can synthesize today, the simpler comprehensions in the second code fragment are.

This research has great potential to make software development a faster and easier task, and to push the bounds of what is possible with program synthesis.

5. REFERENCES

[1] The Bullet physics library. http://bulletphysics.org (Retrieved October 29, 2015).

[2] J. Cai, P. Facon, F. Henglein, R. Paige, and E. Schonberg. Type transformation and data structure choice. In *Constructing Programs From Specifications*, pages 126–124. North-Holland, 1991.

[3] J. Earley. High level iterators and a method for automatically designing data structure representation. *Comput. Lang.*, 1(4):321–342, Jan. 1975.

[4] A. C. Fong and J. D. Ullman. Induction variables in very high level languages. In *Proceedings of the 3rd ACM SIGACT-SIGPLAN Symposium on Principles on Programming Languages*, POPL '76, pages 104–112, New York, NY, USA, 1976. ACM.

[5] S. Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, pages 317–330, New York, NY, USA, 2011. ACM.

[6] P. Hawkins, A. Aiken, K. Fisher, M. Rinard, and M. Sagiv. Data representation synthesis. In *Proceedings of the 32Nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, pages 38–49, New York, NY, USA, 2011. ACM.

[7] S. Jha, S. Gulwani, S. A. Seshia, and A. Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering - Volume 1*, ICSE '10, pages 215–224, New York, NY, USA, 2010. ACM.

[8] Bug list. http://bit.ly/1ZJ4isR (Retrieved June 12, 2016).

[9] C. Loncaric, E. Torlak, and M. D. Ernst. Fast synthesis of fast collections. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2016, pages 355–368, New York, NY, USA, 2016. ACM.

[10] Myria distributed database. http://myria.cs.washington.edu (Retrieved April 10, 2015).

[11] R. Paige and S. Koenig. Finite differencing of computable expressions. *ACM Trans. Program. Lang. Syst.*, 4(3):402–454, July 1982.

[12] Sat4J boolean reasoning library. https://www.sat4j.org (Retrieved February 3, 2016).

[13] E. Schonberg, J. T. Schwartz, and M. Sharir. An automatic technique for selection of data representations in SETL programs. *ACM Trans. Program. Lang. Syst.*, 3(2):126–143, Apr. 1981.

[14] J. T. Schwartz. Automatic data structure choice in a language of very high level. In *Proceedings of the 2Nd ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '75, pages 36–40, New York, NY, USA, 1975. ACM.

[15] A. Solar Lezama. *Program Synthesis By Sketching*. PhD thesis, EECS Department, University of California, Berkeley, Dec 2008.

[16] J. Toman, S. Pernsteiner, and E. Torlak. Crust: A bounded verifier for rust (n). In *Automated Software Engineering (ASE), 2015 30th IEEE/ACM International Conference on*, pages 75–80, Nov 2015.

[17] ZTopo topographic map viewer. https://hawkinsp.github.io/ZTopo/ (Retrieved May 8, 2015).